OFFSIDE LABS

Reflect Core: Yield Routing

Smart Contract Security Assessment

February 2026

Prepared for:

Reflect

Prepared by:

Offside Labs

Ripples Wen

Sirius Xie



Contents

1 About Offside Labs **2**

2 Executive Summary **3**

3 Summary of Findings **7**

4 Key Findings and Recommendations **9**

4.1 Drift market account owner not verified during market lookup 9
4.2 Stale TVL snapshot used for share pricing across venue refresh operations 10
4.3 CapitalConductor transfer execution can sweep controller balance 11
4.4 Oracle validity guard rails not enforced in common pricing paths 12
4.5 Missing user account validation in capture_spread_drift 13
4.6 Slippage check uses user_receipt_ata total balance in mint_strategy1 14
4.7 Missing deposit mint validation in redeem_drift_s3 15
4.8 Incorrect cap check for strategy1 15
4.9 Missing move compression in plan stage 16
4.10 finalise_withdraw_plan overwrites caller-supplied moves 17
4.11 Drift spot mapping helpers iterate unloaded slots 18
4.12 Venue CPI token amount mismatch in strategy1 mint and redeem 19
4.13 Small-leg consolidation may exceed venue balance 20
4.14 Withdraw finalisation ignores post-withdraw ideal caps 20
4.15 Insufficient spot_market_lst account validation in add_lst_drift 21
4.16 Missing account validation for usdc_controller in migrate_s1 22
4.17 Outdated controller deserialization in capture_drift_s1 23
4.18 Conditional mint validation bypass in capture_spread_drift 23
4.19 CapitalConductor basis point computation truncates large ratios 24
4.20 Account size mismatch during controller migration 25
4.21 Jupiter TVL calculation uses non-realtime exchange price 26
4.22 Informational and Undetermined Issues 27

5 Disclaimer **31**

1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.

 **https://offside.io/**

 **https://github.com/offsidelabs**

 **https://twitter.com/offside_labs**

2 Executive Summary

Introduction

Offside Labs completed a security audit of *Reflect Core* smart contracts, starting on December 8th, 2025, and concluding on February 2nd, 2026.

Project Overview

The Reflect Core Yield Routing feature provides a more complete on-chain capability set for routing capital and managing yield across multiple venues. It strengthens the foundational infrastructure for handling accounts and protocol data, and standardizes price and oracle reading as well as related numerical and precision handling. It implements common flows for allocation, rebalancing, yield capture, and distribution across venues. It also strengthens access control and critical input and account consistency checks, and expands error codes and events and logging to improve observability and operations.

Audit Scope

The assessment scope contains mainly the smart contracts of the reflect-main program for the *Reflect Core* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- Reflect Core
  - Codebase: `https://github.com/palindrome-eng/reflect-core`
  - Feature: `tree/feat/yieldRouting`
  - Commit Hash: 5ed9e0e426363de1f6e0c34e096c132f7f98def2

The issues described in this report have been resolved by the following commit:

- Reflect Core
  - Codebase: `https://github.com/palindrome-eng/reflect-core`
  - Commit Hash: 5ff0a16c869fdac0f42ee3249a4a757442ebb7a5

We listed the files we have audited below:

- Reflect Core Yield Routing
  - programs/reflect-main/src/common/constants.rs
  - programs/reflect-main/src/common/derivation.rs
  - programs/reflect-main/src/common/direct_account_operation.rs
  - programs/reflect-main/src/common/errors.rs
  - programs/reflect-main/src/common/events.rs
  - programs/reflect-main/src/common/math/conversion.rs
  - programs/reflect-main/src/common/math/distribution.rs
  - programs/reflect-main/src/common/math/fixed_point.rs
  - programs/reflect-main/src/common/math/lp.rs

- programs/reflect-main/src/common/math/variance.rs
- programs/reflect-main/src/common/oracles/load_maps.rs
- programs/reflect-main/src/common/oracles/oracle_map.rs
- programs/reflect-main/src/common/oracles/pyth/pyth_components.rs
- programs/reflect-main/src/common/program_ids.rs
- programs/reflect-main/src/common/protocols/drift/accounts/fixed_accounts.rs
- programs/reflect-main/src/common/protocols/drift/accounts/perp_market.rs
- programs/reflect-main/src/common/protocols/drift/accounts/spot_market.rs
- programs/reflect-main/src/common/protocols/drift/accounts/user.rs
- programs/reflect-main/src/common/protocols/drift/accounts/user_stats.rs
- programs/reflect-main/src/common/protocols/drift/drift_components.rs
- programs/reflect-main/src/common/protocols/drift/operations/adjust_hedge.rs
- programs/reflect-main/src/common/protocols/drift/operations/calculatons.rs
- programs/reflect-main/src/common/protocols/drift/operations/calls.rs
- programs/reflect-main/src/common/protocols/drift/operations/helpers.rs
- programs/reflect-main/src/common/protocols/drift/spots_drift.rs
- programs/reflect-main/src/common/protocols/jupiter/accounts/fixed_accounts.rs
- programs/reflect-main/src/common/protocols/jupiter/accounts/lending.rs
- programs/reflect-main/src/common/protocols/jupiter/operations/calls.rs
- programs/reflect-main/src/common/protocols/jupiter/operations/exchange.rs
- programs/reflect-main/src/common/protocols/kamino/accounts/fixed_accounts.rs
- programs/reflect-main/src/common/protocols/kamino/accounts/obligation.rs
- programs/reflect-main/src/common/protocols/kamino/accounts/reserve.rs
- programs/reflect-main/src/common/protocols/kamino/operations/calculations.rs
- programs/reflect-main/src/common/protocols/kamino/operations/calls.rs
- programs/reflect-main/src/common/protocols/yield_handler/common.rs
- programs/reflect-main/src/common/protocols/yield_handler/drift_handler.rs
- programs/reflect-main/src/common/protocols/yield_handler/jupiter_handler.rs
- programs/reflect-main/src/common/protocols/yield_handler/kamino_handler.rs
- programs/reflect-main/src/common/seeds.rs
- programs/reflect-main/src/common/token.rs
- programs/reflect-main/src/common/utils.rs
- programs/reflect-main/src/common/verification.rs
- programs/reflect-main/src/instructions/admin/access/action_update.rs
- programs/reflect-main/src/instructions/admin/access/role_holder_update.rs
- programs/reflect-main/src/instructions/admin/components/update_protocol_distribution.rs
- programs/reflect-main/src/instructions/admin/dex/drift/capture_drift/capture_drift_s1.rs
- programs/reflect-main/src/instructions/admin/dex/drift/capture_drift/capture_drift_s3.rs
- programs/reflect-main/src/instructions/admin/dex/drift/capture_drift/capture_entry_drift.rs
- programs/reflect-main/src/instructions/admin/dex/drift/create_user_stats_account.rs

- programs/reflect-main/src/instructions/admin/dex/drift/deposit_drift.rs
- programs/reflect-main/src/instructions/admin/dex/drift/rebalance/process_swap_deposit.rs
- programs/reflect-main/src/instructions/admin/dex/drift/rebalance/process_swap_withdraw.rs
- programs/reflect-main/src/instructions/admin/dex/drift/rebalance/swap_context.rs
- programs/reflect-main/src/instructions/admin/dex/kamino/create_user_accounts.rs
- programs/reflect-main/src/instructions/admin/init_main.rs
- programs/reflect-main/src/instructions/admin/migration/backup.rs
- programs/reflect-main/src/instructions/admin/migration/migrate_s1.rs
- programs/reflect-main/src/instructions/admin/security/freeze_functionality.rs
- programs/reflect-main/src/instructions/admin/security/suspend_spl.rs
- programs/reflect-main/src/instructions/admin/strategies/strategy1/init_controller_s1.rs
- programs/reflect-main/src/instructions/admin/strategies/strategy1/init_drift_accounts_s1.rs
- programs/reflect-main/src/instructions/admin/strategies/strategy1/rebalance_s1.rs
- programs/reflect-main/src/instructions/admin/strategies/strategy3/add_lst_drift.rs
- programs/reflect-main/src/instructions/admin/strategies/strategy3/add_sub_account.rs
- programs/reflect-main/src/instructions/admin/strategies/strategy3/init_controller_s3.rs
- programs/reflect-main/src/instructions/admin/strategies/update_attenuation.rs
- programs/reflect-main/src/instructions/admin/strategies/update_cap.rs
- programs/reflect-main/src/instructions/admin/strategies/update_recipients.rs
- programs/reflect-main/src/instructions/admin/swap/swap_orca.rs
- programs/reflect-main/src/instructions/admin/swap/swap_orca_two_hop.rs
- programs/reflect-main/src/instructions/user/drift/settle/settle_pnl.rs
- programs/reflect-main/src/instructions/user/drift/settle/settle_pnl_multi.rs
- programs/reflect-main/src/instructions/user/strategy1/depracated/mint_drift_s1.rs
- programs/reflect-main/src/instructions/user/strategy1/depracated/redeem_drift_s1.rs
- programs/reflect-main/src/instructions/user/strategy1/depracated/supply_management_context_s1.rs
- programs/reflect-main/src/instructions/user/strategy1/mint_strategy1.rs
- programs/reflect-main/src/instructions/user/strategy1/redeem_strategy1.rs
- programs/reflect-main/src/instructions/user/strategy1/supply_management_strategy_1.rs
- programs/reflect-main/src/instructions/user/strategy3/mint_drift_s3.rs
- programs/reflect-main/src/instructions/user/strategy3/redeem_drift_s3.rs

- programs/reflect-main/src/instructions/user/strategy3/supply_management_context_s3.rs
- programs/reflect-main/src/lib.rs
- programs/reflect-main/src/state/credentials.rs
- programs/reflect-main/src/state/main.rs
- programs/reflect-main/src/state/other/jlp.rs
- programs/reflect-main/src/state/other/jlp_controller.rs
- programs/reflect-main/src/state/other/rebalance_lst.rs
- programs/reflect-main/src/state/other/usdc_controller.rs
- programs/reflect-main/src/state/strategy_controller/base.rs
- programs/reflect-main/src/state/strategy_controller/base_components/access.rs
- programs/reflect-main/src/state/strategy_controller/base_components/action.rs
- programs/reflect-main/src/state/strategy_controller/base_components/attenuation.rs
- programs/reflect-main/src/state/strategy_controller/base_components/capture.rs
- programs/reflect-main/src/state/strategy_controller/base_components/externals.rs
- programs/reflect-main/src/state/strategy_controller/base_components/killswitch.rs
- programs/reflect-main/src/state/strategy_controller/base_components/recipients.rs
- programs/reflect-main/src/state/strategy_controller/base_components/spls.rs
- programs/reflect-main/src/state/strategy_controller/rack.rs
- programs/reflect-main/src/state/strategy_controller/rack_components/autocompound.rs
- programs/reflect-main/src/state/strategy_controller/rack_components/capitalconductor.rs
- programs/reflect-main/src/state/strategy_controller/rack_components/drift_spot_markets.rs
- programs/reflect-main/src/state/strategy_controller/rack_components/drift_sub_accounts.rs
- programs/reflect-main/src/state/strategy_controller/strategy_controller.rs

Findings

The security audit revealed:

- 1 critical issue
- 3 high issues
- 10 medium issues
- 7 low issues
- 10 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.

3 Summary of Findings

ID	Title	Severity	Status
01	Drift market account owner not verified during market lookup	Critical	Fixed
02	Stale TVL snapshot used for share pricing across venue refresh operations	High	Fixed
03	CapitalConductor transfer execution can sweep controller balance	High	Fixed
04	Oracle validity guard rails not enforced in common pricing paths	High	Acknowledged
05	Missing user account validation in capture_spread_drift	Medium	Fixed
06	Slippage check uses user_receipt_ata total balance in mint_strategy1	Medium	Fixed
07	Missing deposit mint validation in redeem_drift_s3	Medium	Fixed
08	Incorrect cap check for strategy1	Medium	Fixed
09	Missing move compression in plan stage	Medium	Fixed
10	finalise_withdraw_plan overwrites caller-supplied moves	Medium	Fixed
11	Drift spot mapping helpers iterate unloaded slots	Medium	Fixed
12	Venue CPI token amount mismatch in strategy1 mint and redeem	Medium	Fixed
13	Small-leg consolidation may exceed venue balance	Medium	Fixed
14	Withdraw finalisation ignores post-withdraw ideal caps	Medium	Fixed
15	Insufficient spot_market_lst account validation in add_lst_drift	Low	Fixed
16	Missing account validation for usdc_controller in migrate_s1	Low	Fixed
17	Outdated controller deserialization in capture_drift_s1	Low	Fixed
18	Conditional mint validation bypass in capture_spread_drift	Low	Fixed
19	CapitalConductor basis point computation truncates large ratios	Low	Fixed
20	Account size mismatch during controller migration	Low	Fixed
21	Jupiter TVL calculation uses non-realtime exchange price	Low	Acknowledged

ID	Title	Severity	Status
22	Incorrect account deserialization offset in get_market_index	Informational	Fixed
23	Incorrect token program account passed to transfer_controller	Informational	Fixed
24	Unvalidated strategy identifier in freeze strategy action event emission	Informational	Fixed
25	Incorrect Kamino obligation v1 field offsets in constant definitions	Informational	Fixed
26	Incorrect market index type used in Drift settle PnL CPI	Informational	Fixed
27	Component data contiguity not preserved when removing rack components	Informational	Fixed
28	Redundant state write-back in rack and strategy instruction paths	Informational	Fixed
29	Spot market to SPL strategy base mapping uses inconsistent index domains	Informational	Acknowledged
30	Token account naming inconsistent with validation logic	Informational	Acknowledged
31	Truncation bias in liquidity total calculation	Informational	Acknowledged

4 Key Findings and Recommendations

4.1 Drift market account owner not verified during market lookup

Severity: Critical	Status: Fixed
Target: Smart Contract	Category: Data Validation

Description

The program resolves Drift spot and perp market accounts by iterating over `ctx.remaining_accounts` and selecting the first account whose data matches an expected 8-byte discriminator and whose decoded `market_index` equals the requested value.

During this process, the account owner is not validated. In particular, the lookup functions accept any account whose data layout matches the Drift market discriminator, regardless of whether the account is owned by the Drift program. As a result, an attacker can supply an arbitrary, attacker-owned account that mimics the Drift market layout and is selected as the market account.

The derived market data is then used to obtain an oracle pubkey, which is subsequently resolved via the oracle loading logic. Although oracle accounts themselves are owner-checked, the market account that points to the oracle is not, allowing market substitution.

Impact

An attacker can influence price selection by substituting a forged market account that references a different, valid oracle account included in the remaining accounts. This causes the program to consume prices for an unintended asset.

User-facing mint and redeem instructions that rely on these prices may compute incorrect minting or redemption amounts. Under favorable price manipulation, this can result in the protocol transferring excess value to the attacker, leading to direct loss of user or protocol funds.

Recommendation

It's recommended to enforce that any account interpreted as a Drift spot or perp market is owned by the Drift program before reading or decoding its data.

Mitigation Review Log

Fixed in commit 95fdcfacf462adfb706a4cfafc4e51308fff952b.

4.2 Stale TVL snapshot used for share pricing across venue refresh operations

Severity: High	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

Strategy1 computes `total_tvl` via `YieldVenue::fetch_all_tvl(...)` and calls `auto_compound.update_pool(total_tvl, ...)` before determining mint/redeem share pricing. The same instruction path can later execute venue CPIs through `transfers.execute(...)`.

For venues where TVL-relevant fields are updated by refresh/claim CPIs, this ordering creates a stale snapshot boundary:

- `mint_strategy1.rs` snapshots `(total_tvl, balances)` and updates pool state before venue-side effects.
- Kamino TVL computation reads reserve/obligation state and derives withdrawable liquidity via exchange-rate/index–dependent fields.
- Kamino `deposit()` / `withdraw()` performs `refresh_reserve` and `refresh_obligation` CPIs before the actual deposit/redeem.

If refresh/claim CPIs change the accounts used by `get_tvl(...)`, the share pricing uses a pre-refresh NAV while the instruction later materializes post-refresh NAV within the same transaction.

Impact

A user can potentially extract value by minting shares when `total_tvl` is understated (pre-refresh) and then redeeming after the refresh boundary when NAV has increased. This is a sandwich-style share dilution vector where yield that should accrue to existing share holders is partially transferred to the attacker.

Recommendation

For each venue that requires refresh/claim to make TVL accurate, perform the required refresh steps before computing `total_tvl`.

Mitigation Review Log

Fixed in commit 96c06cc5dd17bfcd8372252464c6813e4d90d5ca and 82e62747cda2a52be99d0ce8d65f379367a788da.

4.3 CapitalConductor transfer execution can sweep controller balance

Severity: High	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

`CapitalConductor` constructs transfer plans such that, after executing all legs:

- pure rebalances leave controller cash unchanged,
- net deposits leave controller cash near zero,
- net withdrawals leave controller cash equal to the user's withdrawal amount so it can be transferred out.

However, `Transfers::execute` applies a special-case behavior intended to absorb rounding dust: if the last move is a deposit, it deposits the entire controller token balance instead of the planned amount. This behavior is unconditional with respect to the overall net flow direction.

```
47 // On the last move, use whatever's actually left to avoid rounding
    ↪ errors.
48 let actual_amount = if idx == self.moves.len() - 1 && _move.kind ==
    ↪ MoveKind::Deposit {
49     let remaining_balance = controller_balance_before;
50     msg_c!("Last deposit: using actual controller balance {} instead of
        ↪ planned {}",
51            remaining_balance, _move.amount);
52     remaining_balance
53 } else {
54     _move.amount
55 };
```

programs/reflect-main/src/state/strategy_controller/rack_-components/capitalconductor.rs#L47-L55

In net-withdraw plans, this assumption is unsafe. A net withdraw can legitimately include one or more deposit legs (for example, from internal swaps planned before applying the net withdraw). Because moves are reordered such that deposits execute last, the final leg is often a deposit. When this occurs, the "deposit remaining balance" logic can sweep funds that were intended to remain in the controller vault for the user withdrawal.

Separately, the same sweeping behavior can interact poorly with Strategy1 share pricing: controller-held tokens that are not included in the TVL snapshot used by `AutoCompound::update_pool` can later be swept into venues, causing the mint to be priced against an understated baseline.

Impact

- Redeem breakage / DoS: During `redeem_strategy1`, controller cash required to pay the user can be consumed by the final deposit. If `min_usdc_redeem > 0`, the transaction reverts; if it is zero, the user can be underpaid.
- Accounting correctness risk: Transfer plans' cash-flow invariants are violated, making behavior sensitive to internal swap planning and execution order.
- Potential mint mispricing: If non-zero controller idle cash exists at instruction entry and is later swept by the last-deposit rule, shares can be minted using a TVL snapshot that did not include that value, leading to dilution or inconsistent pricing.

Recommendation

It is recommended to remove the "deposit entire remaining balance" behavior and instead rely on explicit, deterministic transfer plan math with well-defined dust handling. If the sweep behavior is retained, gate it strictly to prevent unexpected fund movements and invariant breakage.

Mitigation Review Log

Fixed in commit aa2d13fabaee259e10d19aa6f14427f2aaa39cc8 and 9034d9be37d3af81938a26bd8062c5f53a11f68e.

4.4 Oracle validity guard rails not enforced in common pricing paths

Severity: High	Status: Acknowledged
Target: Smart Contract	Category: Data Validation

Description

The codebase implements an oracle validity policy via `OraclePriceData`, `OracleGuardRails` / `ValidityGuardRails`, and `oracle_validity(...)` (staleness, confidence bounds, volatility, and minimum data points). `OracleMap` exposes `get_price_data_and_validity(...)` to apply these guard rails.

However, common pricing paths obtain oracle prices via `oracle_map.get_price_data(&identifier)?.price`, which returns parsed price data without enforcing the protocol-level validity decision. As a result, oracle prices can be consumed for sizing and valuation even when they violate the configured semantic constraints (e.g., too stale, too wide confidence interval, insufficient data points, or disallowed feed status), depending on the underlying oracle source.

Impact

Strategy sizing and valuation can proceed using stale or low-quality oracle prices. This can misprice user-facing mint/redeem calculations and position/hedge decisions, leading to incorrect outcomes and potential economic loss. The risk increases when attackers can influence which oracle accounts are provided via `remaining_accounts`, because structural parsing checks alone do not guarantee price quality or timeliness.

Recommendation

Add an explicit validity gate at every oracle consumption site.

Mitigation Review Log

Reflect Team: Price is currently only used for event emission, not for any economic calculations. Going forward, a custom price oracle or aggregation of multiple sources is preferable to relying solely on Drift. A dedicated pricing module will be implemented.

4.5 Missing user account validation in capture_spread_drift

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Data Validation

Description

The `capture_spread_drift` instruction accepts a `user_account` parameter but does not validate that it is the expected account for the operation. As a result, a caller can supply an arbitrary account in place of the intended user account.

Impact

An attacker with `Action::Capture` permission can invoke `capture_spread_drift` with a malicious `user_account` and influence how asset deltas are attributed. This enables incorrect accounting and can trigger over-minting of stable tokens. Excess tokens may be issued to recipients, and if the attacker controls any recipient accounts, they can redeem the improperly minted tokens and extract funds from the system.

Recommendation

It's recommended to add strict validation for `user_account` within `capture_spread_drift`.

Mitigation Review Log

Fixed in commit 1efb2e82d2bd3b58a75914d90483ced276c85eec and 135e53f0d8886dd15d997aa6731551a74ee14292.

4.6 Slippage check uses user_receipt_ata total balance in mint_strategy1

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

```
88 // Reject if below the minimum threshold.
89 slippage_pass(token_balance(&accounts.user_receipt_ata.to_account_info())?,
    ↪ min_receipt_receive)
```

programs/reflect-main/src/instructions/user/strategy1/mint_strategy1.rs#L88-L89

In `mint_strategy1`, the slippage check uses the current total token balance of `user_receipt_ata` at the time of the check. The check does not compute the **delta** between the pre-instruction balance and the post-mint/deposit balance. If the user already holds receipt tokens in `user_receipt_ata`, the existing balance is included in the slippage computation.

Impact

A non-zero pre-existing balance in `user_receipt_ata` can cause the slippage check to pass even when the instruction mints fewer receipt tokens than `min_receipt_receive` for the current deposit. This weakens the user protection provided by `min_receipt_receive` and can allow execution under worse-than-expected terms.

Recommendation

Perform slippage validation on the net increase in `user_receipt_ata` for the `mint_strategy1` instruction.

Mitigation Review Log

Fixed in commit 383680783e486f4448c431b249296134f5cbec45.

4.7 Missing deposit mint validation in redeem_drift_s3

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Data Validation

Description

In `redeem_drift_s3`, the instruction does not call `validate_deposit_mint`. As a result, the deposit mint is not checked against the program's allowlist/denylist logic before processing the redemption.

Impact

A user can redeem/withdraw using a suspended SPL mint that should be blocked by policy. This bypasses the intended mint restriction controls and can result in withdrawals of assets that the program treats as suspended.

Recommendation

Enforce deposit mint validation in `redeem_drift_s3`.

Mitigation Review Log

Fixed in commit 95573612493a4a5c7422787ee04620f7cf0a3c99.

4.8 Incorrect cap check for strategy1

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Data Validation

Description

Strategy 1 enforces the cap with the following early-return condition:

```
171 // Happy returns.
172 if action != Action::Mint || base.mintable(mint_supply(receipt_mint)?) >
    ↪ amount {
173     return Ok(());
174 }
```

programs/reflect-main/src/common/verification.rs#L171-L174

`base.mintable(current_supply)` is computed from `cap - current_supply`, where both `cap` and `current_supply` are expressed in USDC+ (receipt mint) units:

```rust
/** Extracts how much more can be minted under this strategy cap. */
pub fn mintable(&self, current_supply: u64) -> u64 {
    if current_supply >= self.cap {
        return 0;
    }
    self.cap.checked_sub(current_supply).unwrap()
}
```

programs/reflect-main/src/state/strategy_controller/base.rs#L154-L160

However, `amount` represents the USDC deposit amount, not the USDC+ amount to be minted. Since the USDC+:USDC exchange rate is generally not 1:1, this check compares values denominated in different units. In addition, the boundary comparison should use `>=` rather than `>` to correctly handle the exact-cap case.

Impact

When the USDC+:USDC rate is greater than 1, deposits that should be allowed under the cap (based on the USDC+ minted for the deposit) can be incorrectly rejected due to the unit mismatch and the strict `>` comparison.

Recommendation

Enforce the cap using consistent units and correct boundary handling.

Mitigation Review Log

Fixed in commit e91be86b3c52e343d15dedc2870b5103038445ec.

4.9 Missing move compression in plan stage

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

In Capital Conductor's plan stage, `plan_net_withdraw` may skip calling `compress_moves` for certain paths. As a result, the generated plan can contain both `withdraw` and `deposit` operations for the same venue within the final move list.

Before execution, moves are reordered to enforce that all `withdraw` operations execute before any `deposit` operations. This reordering can cause a `withdraw` for a venue to execute before the corresponding `deposit` that would have funded it, even when the venue's net position would have been valid after proper compression.

Impact

For venues that reject withdrawals exceeding the current balance, execution can fail when the reordered `withdraw` runs with insufficient funds, causing the transaction to revert. Even for venues that allow such withdrawals, the execution outcome may diverge from the intended plan semantics due to the altered operation ordering.

Recommendation

Ensure that operations are merged for every venue. All `withdraw` and `deposit` operations targeting the same venue should be netted into a single, consolidated operation.

Mitigation Review Log

Fixed in commit 02c1735889db73fc8db144b4592edf1e29079bfe.

4.10 finalise_withdraw_plan overwrites caller-supplied moves

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

In Capital Conductor's plan stage, `finalise_withdraw_plan` assigns a newly computed plan to `moves` by overwriting the vector content:

```
1254 // Build withdrawal caps from starting balances.
1255 let caps = self.compute_withdrawal_caps(starting_balances);
1256
1257 // Allocate withdrawals within those caps.
1258 let new_plan = self.allocate_withdrawals_within_caps(wd, &caps)?;
1259
1260 // Compress and assign back.
1261 *moves = self.compress_moves(&new_plan);
```

programs/reflect-main/src/state/strategy_controller/rack_components/capitalconductor.rs#L1254-L1261

As a result, any prior value of `moves` becomes irrelevant, and the distinctness of earlier control-flow paths is lost because they all converge to the same overwrite behavior.

This also nullifies any caller-side modifications performed immediately before invoking `finalise_withdraw_plan`. For example, the caller pushes additional `UserMove` entries and then calls `finalise_withdraw_plan`, but those pushed entries are discarded due to the unconditional overwrite:

```
798 for (i, amt) in largest_remainders(&weights, wd) {
799     if amt > 0 {
800         moves.push(UserMove {
801             protocol_id: self.protocols[i].id,
802             amount: amt,
803             kind: MoveKind::Withdraw,
804         });
805     }
806 }
807
808 self.finalise_withdraw_plan(moves, starting_balances, wd)
```

programs/reflect-main/src/state/strategy_controller/rack_-components/capitalconductor.rs#L798-L808

Impact

Different withdrawal planning paths that attempt to express different behaviors via `moves` become behaviorally equivalent after `finalise_withdraw_plan`, reducing intended control-flow differentiation.

Recommendation

It's recommended to preserve and incorporate its prior value instead of overwriting it when updating `moves`.

Mitigation Review Log

Fixed in commit 63c7ba05d5833fcff32532188227ea38127d63ba.

4.11 Drift spot mapping helpers iterate unloaded slots

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

The `DriftSpotMarkets` rack component maintains a fixed-size array of spot mappings (`spots: [DriftSpotData; 16]`) and a `spot_count` value that indicates how many entries are initialized. Safe accessors such as `get_loaded_spls()` correctly restrict iteration to `[..spot_count]`.

However, several helper functions iterate over the entire fixed-size array (`lsts.spots.iter()`). This allows unloaded/default slots (typically zero-initialized) to be considered

during lookups, even though they are not part of the active mapping set.

Impact

Iterating over unloaded slots can cause the program to select an incorrect spot mapping, resulting in incorrect `sub_account_id`, `market_index`, or `base_strategy_spl_index` values being used for pricing, deposits, or withdrawals. This can produce unexpected behavior and may lead to economic loss if the wrong market or account is referenced.

Recommendation

Replace all iterations over `lsts.spots.iter()` with `lsts.get_loaded_spls().iter()` so only initialized entries are considered.

Mitigation Review Log

Fixed in commit 9323f08be651b9f62c90de665b2dace7a32837c3.

4.12 Venue CPI token amount mismatch in strategy1 mint and redeem

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

In Strategy1 mint and redeem flows, `Transfer.execute` performs CPIs into external yield venues to execute the actual `deposit` and `withdraw`. For some venues (e.g., Kamino), the amount of tokens actually consumed on deposit or returned on withdrawal can differ from the instruction input `amount` parameter due to venue-side rounding, share/exchange-rate conversions, or internal accounting rules.

If the program assumes the CPI will consume/return exactly the requested amount without verifying the post-CPI deltas, minted/burned share calculations may be based on an expected amount rather than the amount actually transferred.

Impact

A mismatch between requested and executed token amounts can create dust in strategy accounts and distort mint/redeem proportionality and NAV-per-share accounting. Users may receive slightly fewer shares than expected during minting or fewer tokens than expected during redemption, resulting in value loss or inconsistent outcomes across users.

Recommendation

Measure the actual token deltas around venue CPIs and base mint/burn calculations on the executed amounts.

Mitigation Review Log

Fixed in commit aa2d13fabaee259e10d19aa6f14427f2aaa39cc8 and 9034d9be37d3af81938a26bd8062c5f53a11f68e.

4.13 Small-leg consolidation may exceed venue balance

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

At the end of Capital Conductor's plan stage, `consolidate_small_legs` is called to eliminate legs with very small amounts. This consolidation may modify existing moves by increasing certain leg amounts. After this adjustment, a resulting `withdraw` move can become larger than the available balance for its target venue.

Impact

For venues that reject withdrawals exceeding the current balance, execution can fail when the reordered `withdraw` runs with insufficient funds, causing the transaction to revert. Even for venues that allow such withdrawals, the execution outcome may diverge from the intended plan semantics.

Recommendation

When modifies moves, re-validate venue balance constraints for every affected venue.

Mitigation Review Log

Fixed in commit 79ac0df05066c926da7241225da42da4767dc57c.

4.14 Withdraw finalisation ignores post-withdraw ideal caps

Severity: Medium	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

During withdraw finalisation, the implementation builds caps solely from starting balances and allocates withdrawals within those caps:

```
// Build withdrawal caps from starting balances.
let caps = self.compute_withdrawal_caps(starting_balances);

// Allocate withdrawals within those caps.
let new_plan = self.allocate_withdrawals_within_caps(wd, &caps)?;

// Compress and assign back.
*moves = self.compress_moves(&new_plan);
```

programs/reflect-main/src/state/strategy_controller/rack_-components/capitalconductor.rs#L1254-L1261

The logic does not incorporate** post-withdraw ideal caps wh**en finalising the plan. This diverges from the documented behavior, which specifies clamping withdrawals by both starting balances and post-withdraw ideal caps, followed by redistribution to reach the exact withdrawal total.

Impact

Withdrawals can over-drain venues that are already underweight relative to their ideal composition. This breaks intended allocation safeguards and causes behavior to deviate from documentation, which can invalidate downstream assumptions such as risk modeling and capital distribution invariants.

Recommendation

It's recommended to incorporate post-withdraw ideal caps into withdraw finalisation.

Mitigation Review Log

Fixed in commit 4155ffaa5740d34a63d98590c468e83a376b0725.

4.15 Insufficient spot_market_lst account validation in add_lst_drift

Severity: Low	Status: Fixed
Target: Smart Contract	Category: Data Validation

Description

In `add_lst_drift`, the `spot_market_lst` account is only validated by checking that it is owned by the Drift program. The instruction does not validate that the account is a valid spot market account.

Impact

If an incorrect `spot_market_lst` account is provided, the instruction may register an unintended LST configuration. This can cause the program to reference the wrong LST in later flows, resulting in incorrect behavior and persistent state misconfiguration.

Recommendation

Add explicit validation to ensure `spot_market_lst` is a valid Drift spot market account.

Mitigation Review Log

Fixed in commit 2357b3008ef23afce75d6c8b76b06b89e7445135.

4.16 Missing account validation for usdc_controller in migrate_s1

Severity: Low	Status: Fixed
Target: Smart Contract	Category: Data Validation

Description

The `migrate_s1` instruction accepts a `usdc_controller` account but does not validate it.

Impact

If an incorrect `usdc_controller` account is provided, the instruction may deserialize or mutate unintended account data. This can corrupt the state of an unrelated account and can also cause the migration to write invalid configuration/state that breaks program invariants.

Recommendation

Add explicit validation for `usdc_controller` in `migrate_s1` before any reads/writes.

Mitigation Review Log

Fixed in commit 066822570e0015a0c4ec36de85bc57f51c61ef7e and b1c5cca9c941402aa086e17d2def09c0497d4dc0.

4.17 Outdated controller deserialization in capture_drift_s1

Severity: Low	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

After the s1 migration, `capture_drift_s1` still attempts to deserialize the strategy controller account using the legacy `DriftUsdcController` type. However, the post-migration controller account uses a different account type/layout (and therefore a different Anchor account discriminator).

As a result, when `capture_drift_s1` is invoked with a post-migration controller account, the Anchor deserialization (`try_deserialize` / `Account::try_from`) will fail during discriminator validation rather than successfully parsing fields.

Impact

Post-migration controller accounts cannot be used with `capture_drift_s1` . Calls will revert due to discriminator mismatch, effectively making `capture_drift_s1` unusable after the migration.

Recommendation

Migrate the `capture_drift_s1` instruction to support the new controller format.

Mitigation Review Log

Fixed in commit 83279935f9daf23d2eb7d844fbbbe0f4dca096e5.

4.18 Conditional mint validation bypass in capture_spread_drift

Severity: Low	Status: Fixed
Target: Smart Contract	Category: Data Validation

Description

In `capture_spread_drift` , the `stable_mint` account is validated against the expected strategy mint:

```
325 require!(self.mint == mint.key(),
    ↪ ReflectErrorCodes::IncorrectReflectStrategyMint);
```

programs/reflect-main/src/state/strategy_controller/base.rs#L325-L325

However, this validation occurs before the instruction determines whether any stable tokens will actually be minted. If `total_recipient_shares == 0`, the instruction exits early:

```
187 if total_recipient_shares == 0 {
188     return Ok(());
189 }
```

programs/reflect-main/src/state/strategy_controller/base.rs#L187-L189

As a result, the instruction can succeed without enforcing that the provided `stable_mint` is correct when the expected mint amount is zero.

Impact

An attacker can supply a mimic `stable_mint` account and force the expected mint amount to be zero. The instruction will still succeed and emit a `SpreadCapture` event containing incorrect or misleading data. This breaks event correctness guarantees and can mislead off-chain consumers, indexers, or accounting systems that rely on emitted events as a source of truth.

Recommendation

Ensure `stable_mint` validation is enforced regardless of whether minting occurs.

Mitigation Review Log

Fixed in commit a6d8d49fd06a9504f6b2f3c367c80beb7dc0e988.

4.19 CapitalConductor basis point computation truncates large ratios

Severity: Low	Status: Fixed
Target: Smart Contract	Category: Data Validation

Description

`CapitalConductor` computes several decision-critical basis point values using a helper `bps_u128(num, den) -> u16`, defined as `((num * 10_000) / den) as u16`. The calculation is performed in `u128` but is then cast directly to `u16` without capping or checked conversion.

When the computed basis point value exceeds `u16::MAX`, the cast truncates the high bits. As a result, ratios that are mathematically large can be reinterpreted as smaller values after truncation. This helper is used to derive per-leg bump bps for deposits and to gate internal swaps and single-leg routing decisions.

The implementation contradicts the documented definition of `bump_bps` as an unbounded floor division and silently alters semantics for large ratios.

Impact

For deposits, there is no enforced invariant that bounds `deposit_amount` relative to current TVL beyond `TVL != 0`. When `deposit_amount * 10_000 / TVL` exceeds `u16::MAX`, the truncated value can incorrectly satisfy "small bump" leeway checks.

Recommendation

Replace the unchecked cast with a capped or checked conversion.

Mitigation Review Log

Fixed in commit 881b63ff15b449699602bdd5b5336fc75ba95e5a.

4.20 Account size mismatch during controller migration

Severity: Low	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

The protocol uses `migrate_s1` to migrate an existing `DriftUsdcController` account into a `StrategyController`. The migration does not reallocate the account's data size. Previously deployed `DriftUsdcController` accounts were allocated `2555` bytes, while the current `StrategyController` layout expects `10000` bytes.

Impact

If the migrated account remains at `2555` bytes, instructions that assume the `StrategyController` layout size of `10000` bytes may fail due to insufficient account data length or may operate on an incompatible layout, resulting in invalid or inconsistent state.

Recommendation

During `migrate_s1`, reallocate the controller account to the `StrategyController` expected size before writing the new layout, and ensure rent-exemption is maintained for the new size.

Mitigation Review Log

Fixed in commit 498a8dd541ca26d15d6d460210106fc63f52d33e.

4.21 Jupiter TVL calculation uses non-realtime exchange price

Severity: Low	Status: Acknowledged
Target: Smart Contract	Category: Data Validation

Description

```
38 let token_exchange_price =
    ↪ lending_read::token_exchange_price_no_check(lending_data, 1)?;
39
40 let result = (ftokens as u128)
41     .checked_mul(token_exchange_price as u128)
42     .and_then(|v| v.checked_div(EXCHANGE_PRICE_PRECISION))
43     .ok_or(error!(ReflectErrorCodes::MathOverflow))?;
```

programs/reflect-main/src/common/protocols/jupiter/operations/exchange.rs#L38-L43

In `JupiterProtocol::get_tvl`, TVL is computed using the `token_exchange_price` field read from the lending account via `token_exchange_price_no_check`. This exchange price reflects a cached or lagging value stored in the account and does not necessarily represent the current realtime exchange rate derived from Jupiter's latest state and indices.

As a result, the TVL calculation relies on a potentially stale exchange price rather than the up-to-date value used by Jupiter internally.

Impact

TVL used for mint, redeem, or valuation logic can be slightly inaccurate. This can lead to small deviations in share pricing or accounting outcomes, particularly during periods of rapid interest accrual or state updates on the Jupiter side.

Recommendation

Align the TVL calculation with Jupiter's canonical logic by deriving the realtime exchange price instead of using the cached `token_exchange_price` value from the lending account.

Mitigation Review Log

Reflect Team: Fixing this would require replicating Jupiter's internal exchange rate derivation. See `/docs/jupiter-exchange-rate-rounding.md` for our analysis, the effect is bounded and socialised across LPs. We are ok with it.

4.22 Informational and Undetermined Issues

Incorrect account deserialization offset in get_market_index

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Logic Error

In `get_market_index`, the strategy controller account data is deserialized without skipping the first 8 bytes reserved for the account discriminator. Deserialization can interpret the discriminator bytes as part of the struct fields, thus produce incorrect field values.

Fixed in commit 0b05be7b5afbe9129f920f161857cd25e230f679.

Incorrect token program account passed to transfer_controller

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Logic Error

In `create_kamino_user_accounts`, `strategy.transfer_controller` is invoked to transfer USDC to the controller's USDC token account. The instruction passes the `admin` account in the position intended for the SPL Token Program account parameter.

The transfer succeeds because this parameter is not used by `transfer_controller` in the current implementation, so the incorrect account does not affect execution.

Fixed in commit 3a6b6087e42b311a41f2cfe797008a1a83924a32.

Unvalidated strategy identifier in freeze strategy action event emission

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Data Validation

In the `freeze_strategy_action` instruction, the `strategy_id` parameter is not validated against the strategy state/account being acted upon. As a result, the program can emit `StrategyActionUpdate` using a `strategy_id` value that does not correspond to the actual strategy referenced by the instruction accounts.

Fixed in commit 3ef8c2efac164f0a9d12f43cd4c4208761adcd4e.

Incorrect Kamino obligation v1 field offsets in constant definitions

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Logic Error

The `KAMINO_OBLIGATION_V1` constant defines field offsets for `off_borrowed_value_sf`, `off_allowed_borrow_value_sf`, `off_unhealthy_borrow_value_sf`, and `borrow_size` that do not match the actual layout of the Kamino obligation account. Any logic that uses these offsets to parse the obligation account bytes would extract incorrect values.

Fixed in commit eda95cdecf53c91f6ea5a48f4b27436b175025fa.

Incorrect market index type used in Drift settle PnL CPI

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Logic Error

```
297 let mut settle_pnl: [u8; 12] = [0; 12];
298 settle_pnl[0..8].copy_from_slice(&SETTLE);
299 settle_pnl[8..12].copy_from_slice(&(0u32).to_le_bytes());
```

programs/reflect-main/src/common/protocols/drift/operations/calls.rs#L297-L299

```
538 pub fn settle_pnl<'c: 'info, 'info>(
539     ctx: Context<'_, '_, 'c, 'info, SettlePNL>,
540     market_index: u16,
541 ) -> Result<()> {
```

programs/drift/src/lib.rs#L538-L541

The `settle_pnl_drft` helper constructs a CPI to Drift's `settle_pnl` instruction, whose interface uses a `u16` `market_index`. However, the CPI instruction data appends a 4-byte value (`(0u32).to_le_bytes()`), which encodes the market index as a `u32`. This creates an instruction data layout mismatch between the declared parameter type and the serialized payload.

Fixed in commit ff496030838a470189c7aa04a811619e121d7fd5.

Component data contiguity not preserved when removing rack components

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Logic Error

In `Rack::remove_component`, the function removes the corresponding entry from the component registry and shifts subsequent registry entries, but it does not update or compact the underlying component data. As a result, the stored component data can become non-contiguous and no longer match the original contiguous-layout design assumptions.

Fixed in commit 14e001cb175f194071caf147822986df4f90c86b.

Redundant state write-back in rack and strategy instruction paths

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Optimization

Several rack/strategy instruction paths write back to program accounts even when the logical state is unchanged. This results in redundant state mutations and unnecessary account serialization. For examples:

- `add_sub_account` : redundant `insert_strategy_controller`
- `mint_drift_s3` : redundant `update_component` and `insert_strategy_controller`

Fixed in commit e90cc1ba90f00029f29bf96e409eb0f31fce1d34.

Spot market to SPL strategy base mapping uses inconsistent index domains

Severity: Informational	Status: Acknowledged
Target: Smart Contract	Category: Logic Error

`market_index_to_mint` iterates `DriftSpotMarkets.spots` and searches for a matching `market_index` . When it finds a match, it records the array slot position from `enumerate()` as `index` and then uses that value to index into `spls_strategy_base` .

This incorrectly conflates two separate index domains:

- `DriftSpotMarkets.spots` slot index (iteration position in the spots array)
- SPL strategy base index domain represented by `SplBase.index` and referenced by `DriftSpotData.base_strategy_spl_index`

Token account naming inconsistent with validation logic

Severity: Informational	Status: Acknowledged
Target: Smart Contract	Category: Data Validation

```
52 // Validate token accounts.
53 validate_mint_owner(&self.user_usdc_ata.to_account_info(), &USDC_MINT,
   ↪ self.user.key)?;
54 validate_mint_owner(&self.user_receipt_ata.to_account_info(),
   ↪ &self.receipt_mint.key(), self.user.key)?;
```

programs/reflect-main/src/instructions/user/strategy1/supply_management_strategy_1.rs#L52-L54

In `SupplyStrategy1::validate` , `user_usdc_ata` and `user_receipt_ata` are validated only by mint and owner via `validate_mint_owner(...)` . The validation does not enforce that these accounts are the ATA for `(user, mint)` , even though the variables are named with an `_ata` suffix.

Truncation bias in liquidity total calculation

Severity: Informational	Status: Acknowledged
Target: Smart Contract	Category: Precision Issue

```rust
let borrowed = borrowed_sf >> 60;
let protocol_fees = protocol_fees_sf >> 60;
let referrer_fees = referrer_fees_sf >> 60;
let pending_fees = pending_fees_sf >> 60;

let total = available
    .checked_add(borrowed)
    .and_then(|v| v.checked_sub(protocol_fees))
    .and_then(|v| v.checked_sub(referrer_fees))
    .and_then(|v| v.checked_sub(pending_fees))
    .ok_or(ReflectErrorCodes::MathOverflow)?;
```

programs/reflect-main/src/common/protocols/kamino/accounts/reserve.rs#L264-L274

The total liquidity calculation mixes integer liquidity (`available`) with multiple fee components that are individually truncated from fixed-point values using `>> 60` before aggregation. Each fee (`protocol_fees_sf` , `referrer_fees_sf` , `pending_fees_sf`) is converted from a 60-bit fixed-point representation to an integer by right-shifting, which floors the value.

Because truncation occurs independently for each fee term, the final `total` reflects the sum of multiple rounded-down components rather than rounding once on the final result. The computed `total` can be systematically biased upward relative to a calculation that preserves fixed-point precision throughout and truncates only once at the end.

5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

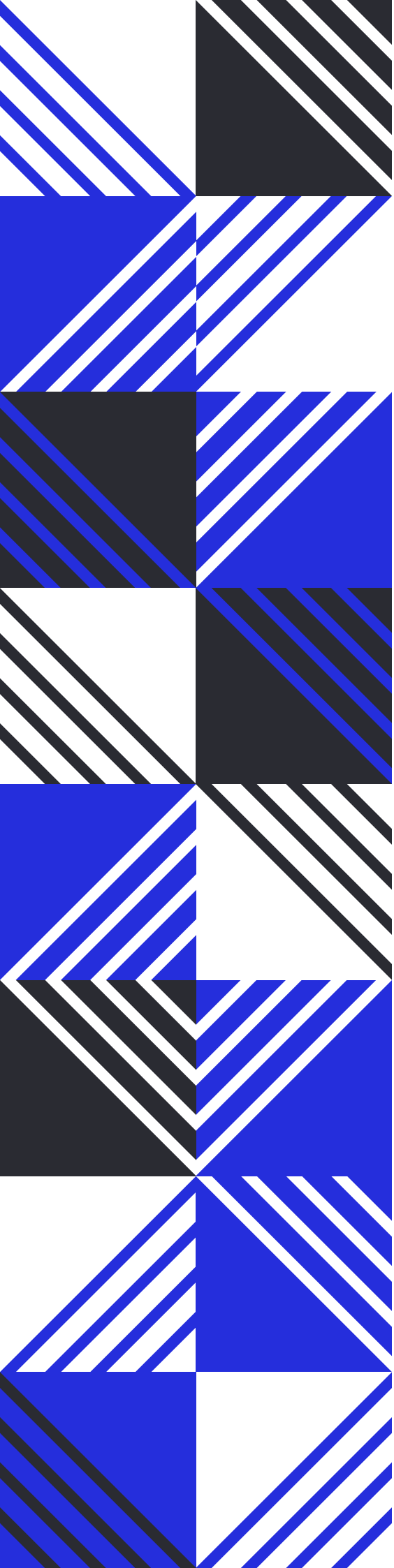
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



OFFSIDE LABS

https://offside.io/

https://github.com/offsidelabs

https://twitter.com/offside_labs